

halfedge：面朝向一致性与可定向性模块设计文档

src/orientation.rs

halfedge 库

2026-07-04

目录

1 模块定位	1
1.1 API 总览	2
2 背景：朝向一致与可定向	2
2.1 朝向一致 (consistently oriented)	2
2.2 可定向 (orientable)	2
3 are_normals_consistent 算法	2
4 is_orientable 算法	3
4.1 思路	3
4.2 与 are_normals_consistent 的区别	3
5 fix_orientations 算法	3
5.1 整体流程	3
5.2 flip_face_orientation: 翻转单个三角面	4
5.3 限制	4
6 复杂度	4
7 退化守卫	4
8 测试覆盖	5
9 设计权衡	5

1 模块定位

orientation.rs 提供面朝向一致性检测与修复能力，回答以下三个问题：

1. 网格中所有相邻面是否朝向一致 (are_normals_consistent)?
2. 网格是否可定向 (is_orientable)?

3. 如何修复不一致的朝向 (fix_orientations)?

1.1 API 总览

类别	API	语义
检测	<code>are_normals_consistent(mesh)</code>	是否所有相邻面朝向一致
检测	<code>is_orientable(mesh)</code>	是否可定向 (无 Möbius 矛盾)
修复	<code>fix_orientations(&mut mesh)</code>	翻转不一致的面, 返回翻转数

2 背景: 朝向一致与可定向

2.1 朝向一致 (consistently oriented)

设半边 h 与其 twin h^* 分别属于面 f_1, f_2 :

$$h: A \rightarrow B, \text{face}(h) = f_1; \quad h^*: B \rightarrow A, \text{face}(h^*) = f_2.$$

朝向一致要求 h 与 h^* 方向相反, 即 $\text{vertex}(h) \neq \text{vertex}(h^*)$ 。

若两者方向相同 ($\text{vertex}(h) = \text{vertex}(h^*)$), 则两面朝向「同向」, 构成局部不一致。

2.2 可定向 (orientable)

可定向是更弱的性质: 网格能否为每个面分配一个朝向, 使所有相邻面朝向一致。不可定向网格 (如 Möbius 带、Klein 瓶) 存在矛盾分配, 无法通过翻转修复。

性质	检查对象	失败原因
一致	当前 mesh 状态	存在同向 twin 对
可定向	BFS 遍历中的方向分配	存在矛盾 (Möbius 结构)

3 are_normals_consistent 算法

遍历所有半边, 检查 twin 对的方向:

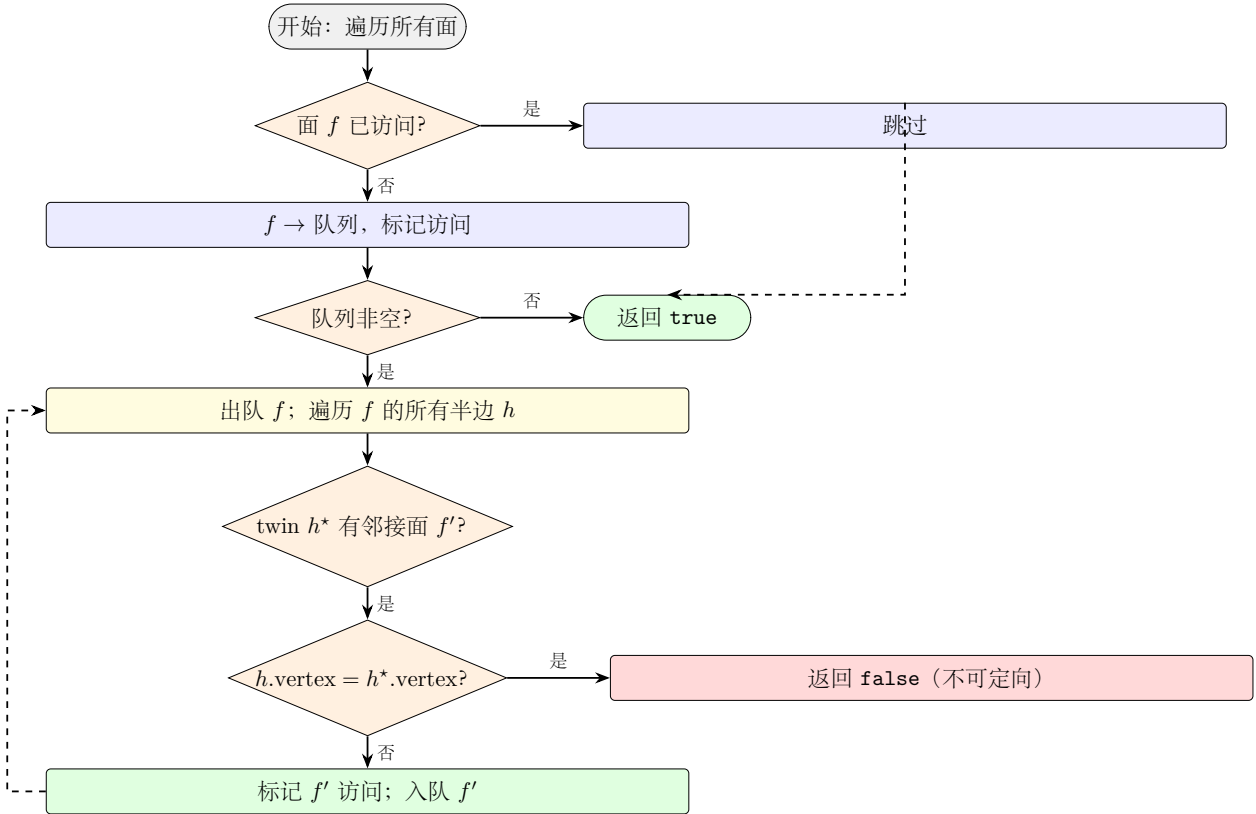
```
1 pub fn are_normals_consistent(mesh: &MeshStorage) -> bool {
2     for he in mesh.halfedge_ids() {
3         let h = match mesh.get_halfedge(he) { Some(h) => h, None => continue };
4         let twin = match h.twin { Some(t) => t, None => continue };
5         let twin_data = match mesh.get_halfedge(twin) { Some(t) => t, None => continue };
6         // he.vertex == twin.vertex ->
7         if h.vertex == twin_data.vertex {
8             return false;
9         }
10    }
11    true
12 }
```

复杂度: $O(E)$ (遍历所有半边), $O(1)$ 额外空间。提前终止: 发现第一处同向即返回 false。

4 is_orientable 算法

4.1 思路

对每个面连通分量做 BFS，从基准面开始传播朝向约束。若某条 twin 边两端方向相同，则不可定向。



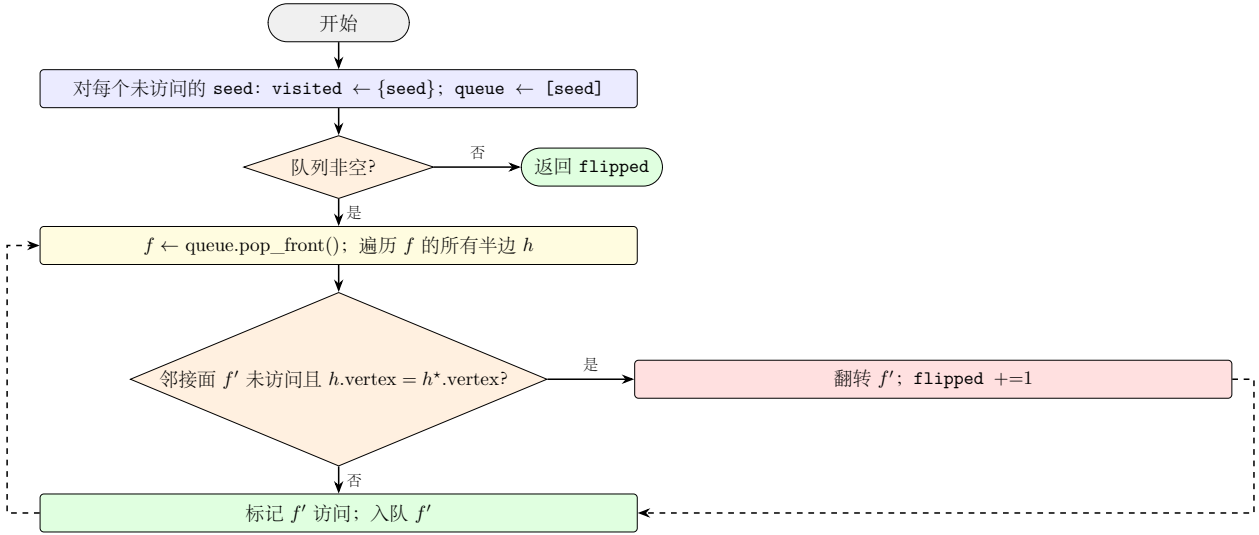
4.2 与 are_normals_consistent 的区别

are_normals_consistent 是当前状态检测，仅判断是否存在同向 twin 对；is_orientable 是可分配性检测，即使当前状态不一致，也可能通过翻转某些面来达到一致（即可定向但当前不一致）。

5 fix_orientations 算法

5.1 整体流程

对每个连通分量，BFS 遍历，遇到与基准方向不一致的邻接面就翻转：



5.2 flip_face_orientation: 翻转单个三角面

对三角面 $[v_0, v_1, v_2]$, 半边环 $h_0 \rightarrow h_1 \rightarrow h_2$, 每条半边指向对应顶点。翻转操作:

1. 交换 **vertex**: $h_0.\text{vertex} \leftarrow v_2, h_1.\text{vertex} \leftarrow v_0, h_2.\text{vertex} \leftarrow v_1$ (每条半边指向原 next 的起点);
2. 交换 **next/prev**: $h_i.\text{next} \leftrightarrow h_i.\text{prev}$ 。

效果: 环顺序由 $v_0 \rightarrow v_1 \rightarrow v_2$ 变为 $v_2 \rightarrow v_1 \rightarrow v_0$ ($\text{CCW} \leftrightarrow \text{CW}$)。

5.3 限制

- 仅支持三角面 ($k = 3$), flip_face_orientation 内部对 $k \neq 3$ 仅交换 next/prev 但不调整 vertex, 可能产生错误结果; 本项目所有网格为三角网格, 故无问题;
- 不可定向网格无法通过翻转达到一致, fix_orientations 不报错, 但剩余矛盾仍存在。调用方应在调用前用 is_orientable 检查。

6 复杂度

API	时间复杂度	空间复杂度
are_normals_consistent	$O(E)$	$O(1)$
is_orientable	$O(V + F)$	$O(F)$ (visited + 队列)
fix_orientations	$O(V + F)$	$O(F)$

7 退化守卫

- are_normals_consistent: get_halfedge 返回 None 时跳过该半边, 不 panic;
- is_orientable: 边界半边 (twin.face = None) 跳过, 不视为矛盾;

- `fix_orientations`: 对 `face_ids().collect::<Vec<_>>()` 先收集所有面 ID，避免借用冲突。

8 测试覆盖

测试名	验证点
<code>icosphere_has_consistent_normals</code>	<code>icosphere(1)</code> 一致且可定向
<code>fix_orientations_noop_on_icosphere</code>	一致网格翻转数 = 0

全模块共 2 个单元测试，覆盖「干净网格」正常路径；项目整体 `cargo test` 共 371 个用例。
待补充：不一致网格翻转修复测试、不可定向网格检测测试（Möbius 构造）。

9 设计权衡

- **BFS 而非 DFS**: 避免栈溢出，对超大连通分量更稳定；
- **仅支持三角面翻转**: 本项目所有网格为三角网格，简化实现；若未来需要支持多边形，需扩展 `flip_face_orientation`；
- **`fix_orientations` 不返回错误**: 不可定向网格无法修复，但本函数仍尝试修复可修复的部分，返回翻转数；调用方可通过后续 `are_normals_consistent` 检查是否完全修复；
- **`visited` 用 `HashSet`**: `FaceId` 实现 `Hash`，查询 $O(1)$ 。